

האוניברסיטה העברית

בי"ס להנדסה ולמדעי המחשב

בחינה בקורסי מבוא למדעי המחשב:

67101 – מבוא למדעי המחשב

67130 – מבוא למדעי המחשב בדגש על תכנות פרוצדוראלי

79115 – מבוא למדעי המחשב - תלפיות

תאריך: 5.6.2009

משך הבחינה: שלוש שעות

מועד ב' - סמסטר א' - תשס"ט – 2008-2009

פרופ' ג'ף רוזנשיין

הנחיות:

1. יש לענות על 3 מתוך 4 השאלות הבאות (3 בלבד). משקל כל השאלות זהה (33 נק').
2. יש לכתוב פתרון לכל אחת מהשאלות שבחרת במחברת נפרדת. יש לציין על כריכת המחברת לאיזה שאלה המחברת מתייחסת, באופן קריא וברור.
3. יש לציין את מס' תעודת הזהות ומספר המחברת על כל מחברת.
4. הקפידו הקפדה יתרה על סדר וכתב יד קריא. בחינה בלתי קריאה לא תזכה את כותבה במלוא הנקודות.
5. יש למחוק טיוטות באופן ברור.
6. יש להשאיר שוליים.
7. אסור להשתמש בעט אדום.
8. הקפידו על סגנון תכנותי נאות: כתיבה באופן מודולרי, ללא שיכפול קוד וכו'.
9. תעדו כל תכנית כך שניתן יהיה להבינה בנקל (מותר לתעד בעברית).
10. פתרון מיטבי לשאלה יחשב פתרון פשוט ויעיל ככל האפשר. תוכנית מסורבלת או מסובכת לא תתקבל כתשובה מלאה (גם אם היא נכונה).
11. אין להקצות או להשתמש במבני נתונים פרט לאלו שהוגדרו בכל שאלה. עם זאת, מותר להשתמש במבני נתונים נוספים אשר גודלם ומספרם אינו תלוי בגודל הקלט (למשל מותר להקצות מערך בן שלשה תאים פעם אחת).
12. בכל השאלות ניתן להניח שהקלט חוקי ומקיים את תנאי השאלה. כמו כן, ניתן להניח כי המערכים והרשימות המקושרות הנתונות מכילים ערכים תקינים ואינם NULL, או מערכים בגודל 0, אלא אם מובהר בשאלה אחרת.

67 - 101
12 / 100

שאלה 1 (33 נקודות)

מספרים רציונליים

ייצוג עשרוני של מספרים במחשב אינו מדויק לגמרי. למשל, המספר $1/3$ בייצוג עשרוני דורש אינסוף ספרות ולכן עשויות להצטבר שגיאות קלות בחישובים במידה ומשתמשים בייצוג עשרוני. מספרים רציונליים הם מספרים הניתנים לייצוג כשבר של שני מספרים שלמים. בשאלה זו עליכם לממש מחלקה המייצגת מספרים רציונליים במדויק. המחלקה תכיל שני שדות פנימיים מטיפוס `int` – מספר אחד עבור המונה ומספר אחד עבור המכנה.

המחלקה תהיה `immutable`: כל הפעולות החשבוניות שביכולתה לבצע מחזירות אובייקטים חדשים ואינן משנות את האובייקטים הקיימים. אין צורך לטפל במקרים בהם גודל המספרים במונה או במכנה חורגים מטווח המספרים הניתן לייצוג ע"י `int`. המחלקה תכיל את השיטות הבאות:

```
public class RationalNum {
    public RationalNum(int numerator, int denominator)
    public RationalNum add(RationalNum other)
    public RationalNum multiply(RationalNum other)
    public RationalNum subtract(RationalNum other)
    public RationalNum divide(RationalNum other)
    public boolean isBiggerThan(RationalNum other)
    public void print()
}
```

השיטות יתוארו בפירוט בהמשך.

א. (12 נק')

ממשו שיטה סטטית פומבית המוצאת את המספר השלם הגדול ביותר המחלק שני מספרים שלמים חיוביים `a, b`.
`public static int maxCommonDivisor(int a, int b)`
 לדוגמה: המחלק המשותף הגדול ביותר של 9 ו-14 הוא 1. המחלק המשותף הגדול ביותר של 100 ו-30 הוא 10.
 שימו לב שניתן לייעל את חיפוש המחלק המשותף הגדול ביותר במידה ונתקלים במחלק משותף קטן יותר ע"י צמצום שני המספרים במחלק הקטן שנמצא.

ב. החלו לממש את המחלקה `RationalNum`:

- (3 נק') רישמו את השדות של המחלקה, ממשו עבורה את הבנאי (constructor) המקבל את המונה והמכנה של המספר הרציונאלי. אין צורך לטפל במקרה שהמכנה הוא 0 (אבל הוא עשוי להיות שלילי).
- (4 נק') ממשו את השיטות `add`, `multiply` (אין צורך לרשום גם את מימוש השיטות `subtract`, `divide` הממומשות באופן דומה אך אסור להסתמך עליהן).
- (2 נק') ממשו את השיטה `isBiggerThan` המשווה בין המספרים ומחזירה אמת אם המספר עליו נקראה השיטה גדול מהמספר הנתון כפרמטר.

היעזרו בשיטה מסעיף א' (גם אם לא עניתם על סעיף זה) כדי לשמור רק שברים מצומצמים במחלקה. שימו לב שהמספרים המיוצגים עשויים להיות גם שליליים.

ג. (12 נק')

ממשו שיטה סטטית פומבית שאינה רקורסיבית

```
public static void findSqrt2(RationalNum precision)
```

המשתמשת במחלקה שכתבתם (ניתן להשתמש בכל השיטות – גם אלו שלא מימשתם) כדי למצוא ולהדפיס מספר רציונאלי קרוב לשורש 2. כדי להדפיס את המספר, השתמשו בשיטה `print()` של המחלקה (אין צורך לממש שיטה זו). המספר המודפס צריך להיות מרוחק לכל היותר עד כדי `precision` מהערך המדויק של שורש 2. כיתבו פתרון יעיל ככל הניתן.

רמז: שורש 2 מצוי בתחום $[1, 2]$ וניתן לבצע חיפוש כדי למצוא אותו.

שאלה 2 (33 נקודות)

עמי ותמי משחקים בגפרורים. עמי נותן לתמי n גפרורים ותמי מחזירה לעמי גפרורים בתורות לפי הכללים הבאים:

1. אם בידי תמי מספר זוגי של גפרורים, היא יכולה להחזיר לעמי בדיוק מחצית מהגפרורים שבידה.
 2. אם מספר הגפרורים בידי תמי מתחלק ב-3 או ב-4 (ללא שארית), ובידיה לפחות 10 גפרורים, תמי יכולה להחזיר גפרורים כמספר המתקבל ממכפלת שתי הספרות האחרונות של מספר הגפרורים שבידה. למשל, אם יש לתמי 342 גפרורים $[114=342/3]$ אזי תמי יכולה להחזיר שמונה $(2*4)$ גפרורים.
 3. אם מספר הגפרורים בידי תמי מתחלק ב-5 (ללא שארית) אזי היא יכולה להחזיר 42 גפרורים בדיוק (אם יש לה יותר מ-42).
- במידה ומתקיים יותר מתנאי אחד תמי יכולה לבחור על פי איזה תנאי לפעול. תמי מנצחת אם היא הצליחה להשאיר בידיה בדיוק 42 גפרורים.

דוגמה למהלך משחק:

עמי נותן לתמי 250 גפרורים.
250 מתחלק ב-5 ותמי מחזירה לו 42 ונשארים לה 208
208 הוא מספר זוגי ולכן תמי מחזירה חצי מהגפרורים ונשארת עם 104 גפרורים.
תמי שוב מחזירה חצי מהגפרורים ונשארת עם 52 גפרורים.
52 מתחלק ב-4 ותמי יכולה להחזיר $10=2*5$ גפרורים. בכך היא נשארת עם 42 גפרורים ומנצחת.

כתבו שיטה רקורסיבית `public static boolean tami(int n)` המחזירה `true` ומדפיסה את סדרת הצעדים המובילה לניצחון אם תמי יכולה לנצח במשחק שמתחיל עם n גפרורים. אם יש כמה סדרות צעדים אפשריות הדפיסו אחת מהן. במידה ותמי אינה יכולה לנצח השיטה תחזיר `false` ולא תדפיס דבר.
מותר להקצות עד $O(n)$ זכרון.

הפלט עבור הדוגמה לעיל (כלומר לאחר הקריאה `tami(250)`) הוא:

42

104

52

10

שאלה 3 (33 נקודות)

נתון מערך של אובייקטים מטיפוס `Student`. לאובייקטים מטיפוס זה יש שיטה: `public int getAge()`. המחזירה את גילו של הסטודנט. בשאלה זו נממש אלגוריתם `ternaryQuicksort` בכדי למיין את הסטודנטים ע"פ גילם בסדר לא יורד (כלומר מערך ממוין בסדר עולה פרט לערכים חוזרים). האלגוריתם עובד כך: ראשית, בוחרים שני איברי ציר (`pivots`) מתוך המערך. לאחר מכן מסדרים את המערך כך שהוא מחולק לשלושה חלקים (לא ממוינים): לפני איבר הציר הראשון מופיעים כל הסטודנטים שגילם גדול או שווה לזה של איבר הציר הראשון וקטן מזה של איבר הציר השני. לאחר איבר הציר השני מופיעים כל הסטודנטים שגילם גדול או שווה לזה של איבר הציר השני. שימו לב כי לאחר סידור זה שני איברי הציר נמצאים במקומם הסופי במערך. כעת ממיינים רקורסיבית את שלושת חלקי המערך המופרדים ע"י ערכי הציר.

א. (20 נק')

כתבו שיטה לא רקורסיבית:

```
public static int[] ternaryPartition(Student[] data,
                                     int lower, int upper)
```

המסדרת את קטע המערך `data` שבין האינדקסים `lower` ו-`upper` כפי שתואר לעיל. מותר לקרוא לשיטה `getAge` של כל סטודנט במערך רק פעם אחת. השיטה מחזירה מערך בן שני תאים המכיל את האינדקסים של איברי הציר במערך לאחר הסידור הנ"ל.

על מנת לבחור את איברי הציר, יש להשתמש בשיטה הבאה (לא צריך לממש שיטה זו):

```
public static int[] getPivots(Student[] data,
                              int lower, int upper)
```

השיטה מחזירה מערך בן שני תאים המכיל את האינדקסים של שני איברי הציר הנבחרים בתוך קטע המערך `data` שבין האינדקסים `lower` ו-`upper`. ניתן להניח שהערך המתאים לתא 0 של המערך המוחזר אינו גדול מהערך המתאים לתא 1. לדוגמה, בהינתן מערך המכיל סטודנטים בגילאים הבאים:

7	25	12	15	21	5	10	25	7	11
---	----	----	----	----	---	----	----	---	----

אם השיטה `getPivots` מחזירה את האינדקסים 2 ו-10, אזי לאחר קריאה ל-`ternaryPartition` המערך יכול להיראות כך:

5	7	11	7	10	12	15	25	21	25
---	---	----	---	----	----	----	----	----	----

המערך המוחזר ע"י `ternaryPartition` במקרה זה יכיל את האינדקסים 1 ו-5.

ב. (13 נק')

כתבו שיטה:

```
public static void ternaryQuicksort(Student[] data)
```

הממיינת בצורה רקורסיבית את המערך הנתון בסדר לא יורד כמתואר לעיל. ניתן להניח שהשיטה `ternaryPartition` עובדת כהלכה גם אם לא פתרם את סעיף א'.

שאלה 4 (33 נקודות)

מימוש "תור קדימויות" באמצעות רשימה מקושרת

"תור קדימויות" הינו מבנה נתונים המאפשר הכנסה, שליפה, ועדכון עדיפויות של ערכים לפי הכללים הבאים:

1. כאשר מכניסים ערך לתור יש להגדיר גם את עדיפותו (מספר שלם).
2. כאשר נדרש להוציא את הערך הבא בתור, יש להוציא את הערך בעל העדיפות הגבוהה ביותר. (אם ישנו יותר מערך אחד כזה, יש להוציא ערך אחד בעל העדיפות הגבוהה ביותר, באופן שרירותי.)

דוגמה:

אם ערכי התור הם מחרוזות, והוכנסו לתור המחרוזות הבאות (כאשר בסוגריים מצוינות עדיפויותיהן):
Moshe(8), Tel-Aviv(3), David(4), Jerusalem(3), Netanya(9)
לאחר מכן, שונתה עדיפותה של המחרוזת Jerusalem ל-10.
אזי, כשנוציא מהתור 5 ערכים בזה אחר זה, יוצאו הערכים הבאים (משמאל לימין):
Jerusalem, Netanya, Moshe, David, Tel-Aviv

נתונה המחלקה הבאה לייצוג חוליה ברשימה מקושרת:

```
public class Node {
    public String data;
    public int priority;
    public Node next;
    public Node(String nData, int nPriority, Node nNext){...}
}
```

כל חוליה ברשימה מכילה מחרוזת (data) ועדיפות (priority).
המחלקה PriorityQueue מנהלת תור קדימויות של מחרוזות:

```
public class PriorityQueue {
    public PriorityQueue()
    public void enqueue(String data, int priority)
    public String dequeue()
    public void changePriority(String data, int newPriority)
    public boolean isEmpty()
}
```

עליך לממש את שיטות המחלקה PriorityQueue, תוך ייצוג התור כרשימה מקושרת של אובייקטים מהמחלקה Node, לפי הפירוט הבא:

- א. בנאי PriorityQueue היוצר תור קדימויות חדש (ריק) (2 נק').
- ב. השיטה enqueue שמטרתה להכניס לתור מחרוזת עם עדיפותה, בסיבוכיות $O(n)$ (כאשר n הוא מספר הערכים בתור) (12 נק').
- ג. השיטה dequeue שמטרתה להוציא מהתור את המחרוזת הבאה בתור (לפי הכללים שהוגדרו לעיל) בסיבוכיות $O(1)$. אם התור ריק, על השיטה dequeue להחזיר null (5 נק').
- ד. השיטה changePriority שמטרתה לשנות את עדיפותה של המחרוזת data בתור, בסיבוכיות $O(n)$. אם המחרוזת data אינה קיימת בתור, השיטה לא תבצע דבר. אם המחרוזת data מופיעה בתור יותר מפעם אחת, השיטה תשנה את עדיפות אחד המופעים של מחרוזת זו בתור (12 נק').
- ה. השיטה isEmpty שמטרתה להחזיר ערך boolean המציין האם התור ריק או לא, בסיבוכיות $O(1)$ (2 נק').

הנך רשאי להוסיף למחלקה PriorityQueue שיטות פרטיות ו-data-members כרצונך.

ב ה צ ל ה !